

Credit Card Default Prediction Summary

1. Data Preparation

The dataset (UCI/Kaggle "default of credit card clients," ~30,000 records) required light cleaning rather than heavy repair: no missing values and no duplicate rows were found. Cleaning focused on aligning the data with its documented dictionary:

- **EDUCATION**: the dictionary defines codes 1–4, but the data also contained undocumented/duplicate codes 0, 5, and 6 (all effectively "unknown"). These were consolidated into 4 ("others").
- **MARRIAGE**: an undocumented code 0 (54 records) was folded into 3 ("others").
- **PAY_0** → **PAY_1**: renamed for consistency with **BILL_AMT1–6** and **PAY_AMT1–6**, which all start at 1.
- **ID** was dropped (a label, not a predictor), and the target was renamed to **PMT_DEFAULT**.
- The undocumented 2 / 0 codes present in every **PAY_*** column were left as-is rather than guessed at, since they represent a large share of records and their true meaning isn't stated anywhere.

EDA confirmed the dataset is **imbalanced** (~22% default rate) and surfaced two strong early signals: default rate falls steadily as **LIMIT_BAL** rises, and jumps sharply once **PAY_1** shows any late payment — both later confirmed as top predictors.

Feature engineering compressed the six months of repayment/billing history into summary features rather than leaving the model to work with 18 separate monthly columns:

- **AVG_BILL_AMT**, **AVG_PAY_AMT**, **TOTAL_PAY_AMT** — spending/payment level
- **PAY_RATIO_i** (and **AVG_PAY_RATIO**) — fraction of each prior bill actually paid off, a direct behavioral signal (undefined 0/0 cases filled with 0)
- **MAX_DELAY**, **AVG_DELAY**, **MONTHS_DELAYED**, **EVER_DELAYED** — how often and how severely a client fell behind
- One-hot encoding of **SEX**, **EDUCATION**, **MARRIAGE** (categorical, not ordinal)

Impact on performance: the engineered features weren't just noise — **MONTHS_DELAYED** and **EVER_DELAYED** ranked in the **top 3 features by importance/coefficient in all three models**, ahead of most raw columns, and **AVG_PAY_RATIO** / **TOTAL_PAY_AMT** also appeared in the Decision Tree's top 5. This indicates that summarizing repayment *patterns* (how consistently, not just how much) gave the models more predictive signal than the raw monthly columns alone would have.

2. Models Training

Three models were built, all trained on the same scaled, feature-engineered data (scaling via **StandardScaler**, needed for the two linear models and applied to the tree as well for consistency):

- **Logistic Regression** — estimates the probability of default as a weighted, linear combination of features passed through a sigmoid. Chosen as an interpretable baseline classifier well-suited to a binary outcome; its coefficients directly show each feature's direction and relative influence.
- **Ridge Regression** — ordinary linear regression with L2 regularization, which shrinks coefficients to control overfitting (tuned via **GridSearchCV** over **alpha**). It's technically a regressor, so its continuous output was thresholded to produce a classification. Included to see whether regularization would improve generalization over plain logistic regression, and as a contrast case for how shrinkage affects a class-imbalanced, recall-sensitive problem.

- **Decision Tree** — a rule-based model that recursively splits on feature thresholds (`max_depth` , `min_samples_leaf` , and `criterion` tuned via `GridSearchCV`). Chosen because it can capture **non-linear interactions and threshold effects** — e.g., a repayment delay past a certain point mattering much more than a linear model can express — which is plausible for repayment-status data like `PAY_1` .

All three models used a **tuned decision threshold** (via `TunedThresholdClassifierCV` , or an equivalent out-of-fold F1 scan for Ridge) rather than the default 0.5 cutoff, since the ~22% default rate makes accuracy-optimized thresholds biased toward the majority class. Notably, combining a lowered threshold with class weighting was tried and rejected — the two corrections stacked and overcorrected, actually hurting F1 — so threshold tuning alone was used to trade precision for recall.

3. Model Performance & Selection

Model	Accuracy	Precision	Recall	F1	Defaults Caught	Defaults Missed
Logistic Regression	0.760	0.466	0.577	0.515	957	702
Ridge Regression	0.770	0.483	0.558	0.518	926	733
Decision Tree	0.765	0.475	0.585	0.524	971	688

- **Logistic Regression** was the most interpretable but the most rigid, limited to a linear decision boundary.
- **Ridge Regression** achieved the best accuracy and precision, but at the cost of the **lowest recall** — its coefficient shrinkage made it more conservative about flagging risk, missing the most actual defaulters (733). For this problem, that's the wrong trade-off.
- **Decision Tree** won on both F1 and recall, catching 971 of 1,659 true defaulters in the test set (the most of any model) while missing the fewest (688). Its ability to split non-linearly on `PAY_1` and the engineered delay features let it draw a tighter boundary around high-risk clients than either linear model could.

Final choice: Decision Tree. Because a missed defaulter (a client who keeps getting credit extended unflagged) is more costly than a false alarm, recall was weighted alongside F1 in the selection criteria rather than optimizing for accuracy or precision alone — which is exactly why Ridge, despite its higher accuracy, was passed over. The Decision Tree's combination of the highest F1 *and* the highest recall made it the clear winner on both criteria simultaneously.

It's worth noting the F1 of 0.524 is modest in absolute terms. This is plausible given the dataset: the records span April–September 2005, during Taiwan's credit card/cash-card debt crisis, when repayment behavior was likely less stable than in normal periods, and the data lacks cross-institution debt exposure or income — both of which would likely raise the ceiling on any model built from this single bank's records alone.